

Python Quiz Practice

Programming for AI — Quiz Practice Set

Topics: I/O, Operators, Data Types, Conditionals, Loops, Functions, Lambda, Map/Filter

Section A: Critical-Thinking Coding Problems (Write the code yourself)

- Q1.** Write a program that takes a number as input and checks whether it is a **palindrome** (reads the same forward and backward) *without converting it to a string*. Use only arithmetic operators.
- Q2.** Take a sentence as input from the user. Using loops and conditionals only (no built-in `.count()`), count how many vowels and how many consonants it contains.
- Q3.** Write a function `classify(n)` that returns "Fizz" if `n` is divisible by 3, "Buzz" if divisible by 5, "FizzBuzz" if divisible by both, and the number itself otherwise. Then use a loop to print `classify(n)` for `n` from 1 to 30.
- Q4.** Write a function that takes a list of numbers and returns a **new list** containing only the numbers that are both even *and* greater than the average of the list. (Hint: you'll need to compute the average first, then filter.)
- Q5.** Using a **lambda function** combined with `filter()`, extract all the strings from a mixed list that have more than 4 characters. Example input: `data = ["cat", "elephant", 12, "dog", "python", 5, "ai"]`
- Q6.** Using `map()` and a lambda function, convert a list of temperatures in Celsius to Fahrenheit. Formula: $F = (C * 9/5) + 32$.
- Q7.** Write a user-defined function `is_prime(n)` that returns `True / False`. Then use `filter()` with this function (not a lambda) to print all prime numbers between 1 and 50.
- Q8.** Write a program that asks the user to enter numbers one at a time (using a loop) until they type `-1`. Use conditionals to keep track of and finally print the **maximum**, **minimum**, and **sum** of all numbers entered (excluding -1).
- Q9.** Without using any built-in reverse function, write a function `reverse_list(lst)` that returns the reverse of a list using a loop.
- Q10.** A store gives discounts based on purchase amount using these rules (use `if-elif-else`): - Above 10000 → 20% discount - 5000-10000 → 10% discount - 1000-4999 → 5% discount - Below 1000 → no discount

Write a function `calculate_discount(amount)` implementing this, then test it with 5 different values.

- Q11.** Using nested loops, print the following pattern for `n = 5`:

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

- Q12.** Write a lambda function that takes two arguments and returns the larger one. Then use it inside a loop to find the maximum number in a list of 5 numbers (without using `max()`).
-

Section B: Predict the Output (Think before running!)

Q13.

```
x = 5
y = "5"
print(x == y)
print(x == int(y))
print(x is int(y))
```

Q14.

```
def func(a, b=10):
    return a + b

print(func(5))
print(func(5, 20))
print(func(b=1, a=2))
```

Q15.

```
nums = [1, 2, 3, 4, 5]
result = list(map(lambda x: x**2 if x % 2 == 0 else x, nums))
print(result)
```

Q16.

```
a = 10
b = 3
print(a // b)
print(a % b)
print(a ** b)
print(not(a > b) and (b > 0 or a < 0))
```

Q17.

```
def outer():
    x = 1
    def inner():
        x = 2
        print("inner:", x)
    inner()
    print("outer:", x)

outer()
```

Q18.

```
lst = [1, 2, 3, 4, 5, 6]
evens = list(filter(lambda x: x % 2 == 0, lst))
squared = list(map(lambda x: x*x, evens))
print(squared)
print(sum(squared) > 50)
```

Q19.

```
i = 0
while i < 5:
    if i == 3:
        i += 1
        continue
```

```
print(i)
i += 1
```

Section C: Find the Error (Identify and explain what's wrong — then fix it)

Q20. Spot the errors in the following code snippets (there may be more than one error per snippet):

Snippet 1:

```
def greet(name)
    print("Hello " + name)

greet("Ali")
```

Snippet 2:

```
x = 10
y = 0
if x > 5
    print("x is large")
else:
    print("x is small")
```

Snippet 3:

```
nums = [1, 2, 3, 4]
squared = map(lambda x: x*x, nums)
print(squared[0])
```

Snippet 4:

```
def add(a, b):
    return a + b

result = add(5, "10")
print(result)
```

Snippet 5:

```
for i in range(1, 10)
    print(i)
```

Snippet 6:

```
x = 5
if x = 5:
    print("x is five")
```

Snippet 7:

```
def calculate_area(radius):
    pi = 3.14
    area = pi * radius ** 2
    print(area)

calculate_area(5)
```

Answer Key

Q1 (sample logic): Reverse the number using `% 10` and `// 10` in a loop to build the reversed number, then compare with original.

Q3: Output for $n=1..30$ follows standard FizzBuzz logic — spot check: $n=15 \rightarrow \text{“FizzBuzz”}$, $n=9 \rightarrow \text{“Fizz”}$, $n=10 \rightarrow \text{“Buzz”}$, $n=7 \rightarrow 7$.

Q5: `["elephant", "python"]`

Q6: Applies formula element-wise via `map`.

Q7: Primes: 2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47.

Q13:

```
True
True
False
```

(`x == y` is actually `False` — 5 is int, “5” is str, they’re never equal. `x == int(y) → True`. `x is int(y) → False`, since `is` checks identity not value, and small-int caching doesn’t guarantee `is` for computed values reliably — safest answer: `False`.) *Correction:* Line 1 prints `False`, not `True` — this is a common trap question. Always compare types before assuming equality.

Q14:

```
15
25
3
```

Q15: `[1, 4, 3, 16, 5]` (evens get squared, odds stay same)

Q16:

```
3
1
1000
False
```

`(not(10>3) = not True = False ; False and (...)) = False`

Q17:

```
inner: 2
outer: 1
```

(local scope inside `inner()` doesn’t affect `outer()`’s `x`)

Q18: Evens = `[2,4,6]` → squared = `[4,16,36]` → sum = 56 → `True`

```
[4, 16, 36]
True
```

Q19:

```
0
1
2
4
```

(when `i==3`, it increments and `continue`s, skipping the `print` for 3)

Q20 Errors: - **Snippet 1:** Missing colon after `def greet(name)` . - **Snippet 2:** Missing colon after `if x > 5` . - **Snippet 3:** `map()` returns a map object, not subscriptable directly — must wrap in `list()` first: `list(squared)[0]` . - **Snippet 4:** No syntax error, but this causes a **TypeError** at runtime: can't add `int + str` . Must convert: `add(5, int("10"))` . - **Snippet 5:** Missing colon after `range(1, 10)` . - **Snippet 6:** `=` is assignment, not comparison — should be `==` . Also causes `SyntaxError` in Python (assignment not allowed in `if` this way). - **Snippet 7:** `area` is a local variable inside `calculate_area()` ; the `print(area)` outside the function (note the indentation — it's actually inside due to missing dedent, but even so) will raise a **NameError** since `area` isn't returned or global. Fix: `return area` and print the returned value outside.

Tip for tomorrow: FAST instructors love combining topics — like using loops with lambda logic, or nested conditionals inside functions. Also double check operator precedence (like in Q16) and scope-related traps (Q17) since these come up a lot in “predict the output” style questions.